Solving a problem in the UK Real Estate Industry using the Wolfram Language.

The Problem

By law in the UK, you must be provided with an Energy Performance Certificate (EPC) before you purchase or rent a property. Many property search websites list the “EPC graph” which gives you a fairly good idea of how energy efficient a property is.

However, these graphs are provided as image files in only an approximately standard format. This makes parsing them with standard OCR techniques tricky.

The market would be more efficient if these energy ratings were shown on the list of listings, so that people don't waste their time looking at properties which are poorly insulated (which in the UK, is most of them).

In[7]:=
```
Get["./m_functions_.m"];
```

In[6]:=
```
SetDirectory[NotebookDirectory[]];
```

In[8]:=
```
Quiet@SetDirectory["./05_cluster_medians/"];
```

In[9]:=
```
FileNames["*.png"]
```

Out[9]=
```
{01_c1_509_467_.png, 02_c2_275_258_.png, 03_c3_336_315_.png, 05_c5_719_676_.png}
```

We start with a representative image from {cluster1, cluster2, cluster3, cluster5 }

Out[10]=

#1 Cropping of Edges

In[11]:=

```
output0 = ImageCrop /@ fils
```

Out[11]=

Resizing to 500x500

In[12]:=

```
output1 = ImageResize[#, {500, 500}] & /@ (ImageResize[#, {509, 467}] & /@ output0)
```

Out[12]=

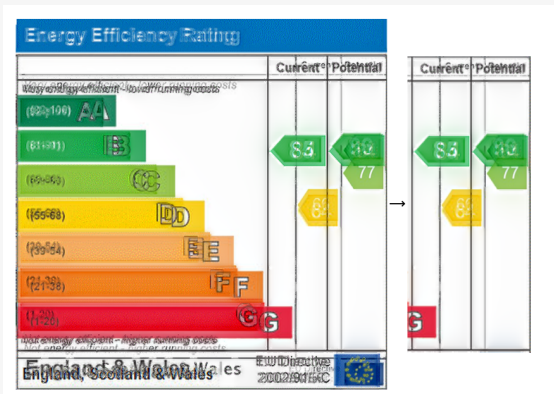
In[66]:=

```
ImageDimensions /@ output1
```

Out[66]=

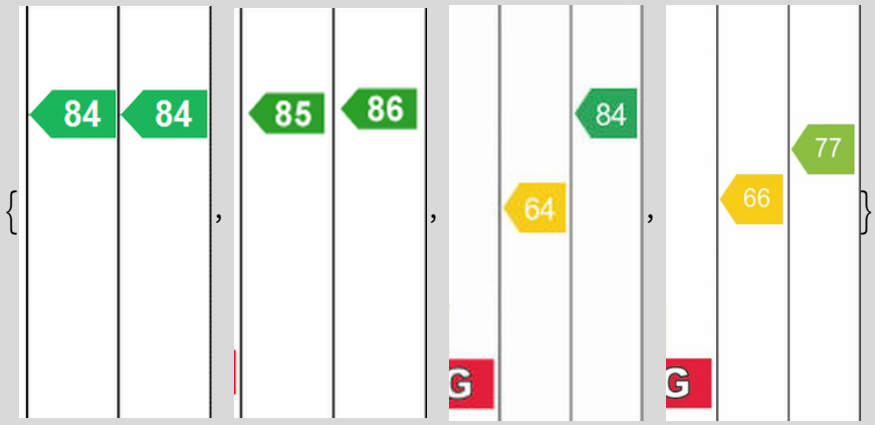
```
{{500, 500}, {500, 500}, {500, 500}, {500, 500}}
```

Trimming based on superposition



In[13]:=
```
trimmer[im_] := Module[{x, y}, {x, y} = ImageDimensions[im];
    ImageTrim[im, ({{2 / 3 x, 1.2 / 10 y}, {1 x, 8.3 / 10 y}})]];
```

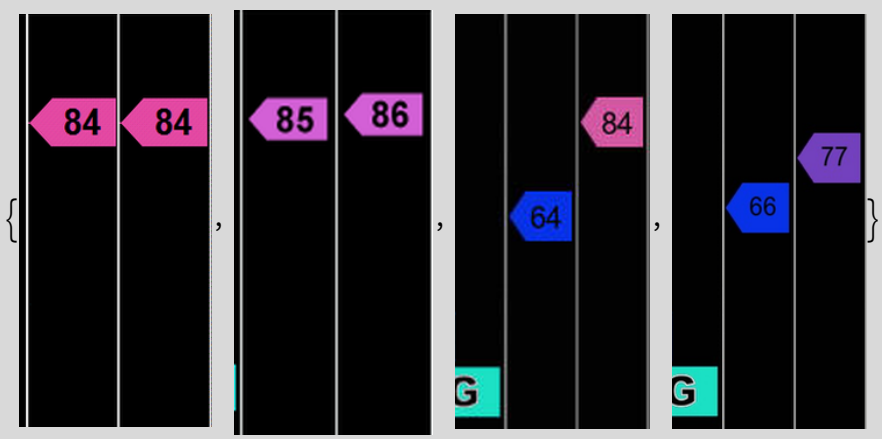
Out[14]=



Out[67]=
```
{{167, 357}, {167, 357}, {167, 357}, {167, 357}}
```

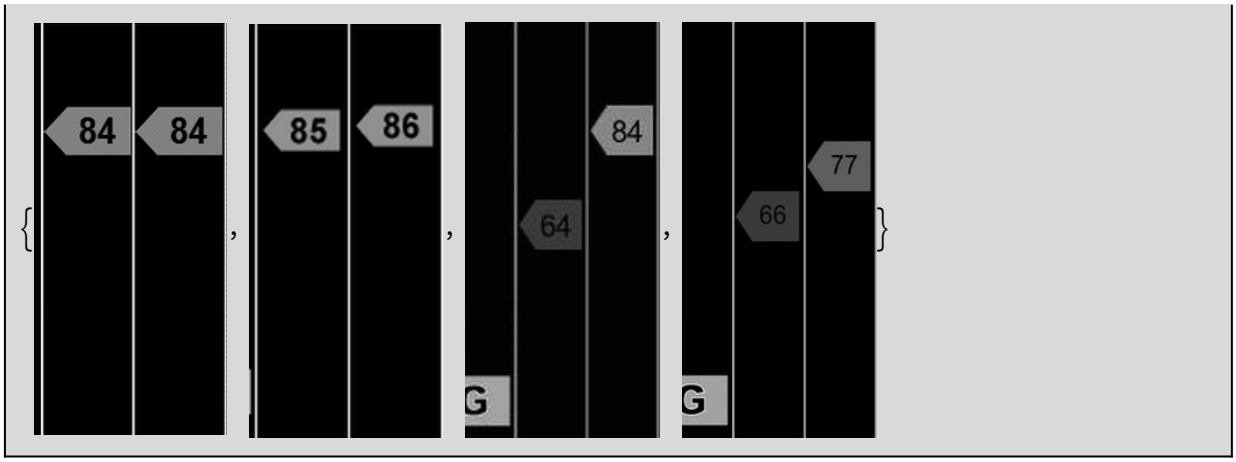
photographic negative

Out[15]=



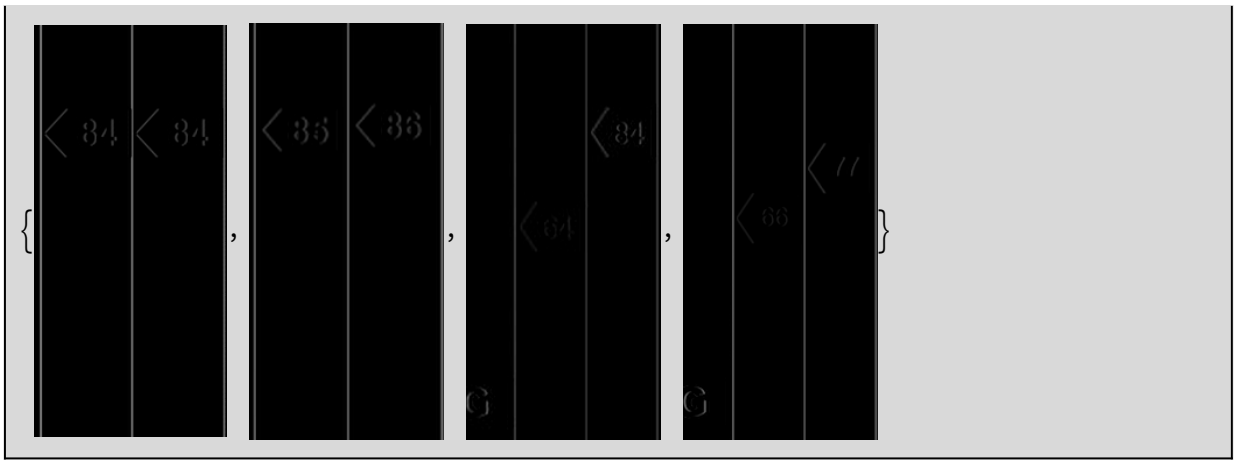
Converting to grayscale

Out[16]=



Blurring with a Gaussian Filter to help with line detection

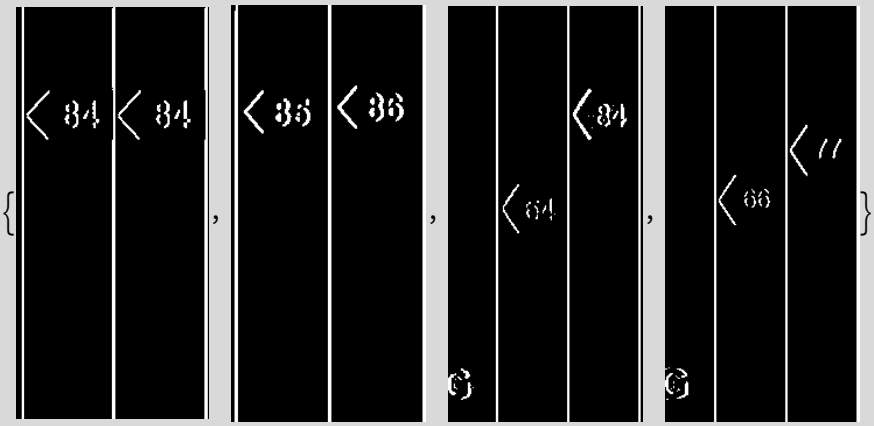
Out[17]=



Binarizing (making all pixels either black or white)

In[18]:=
```
output43 = Binarize[#, 0.05] & /@ output42
```

Out[18]=



Detecting infinite lines using RANSAC method

Out[19]=
```
{Line[{{85.2375 , 357.}, {83.4525 , 0.}}],
 Line[{{165.146 , 357.}, {163.469 , 0.}}], Line[{{5.45768 , 357.}, {6.5768 , 0.}}]}
```

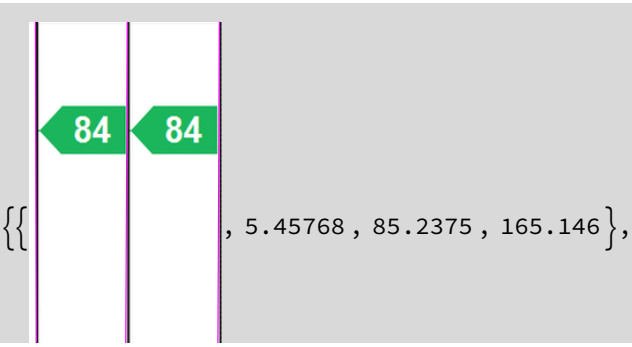
filtering out vertical lines

Out[20]=
```
{Line[{{85.2375 , 357.}, {83.4525 , 0.}}],
 Line[{{165.146 , 357.}, {163.469 , 0.}}], Line[{{5.45768 , 357.}, {6.5768 , 0.}}]}
```

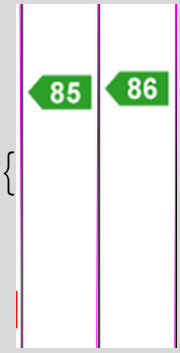
highlighting line location on cropped images

vertical line filter is hardly necessary

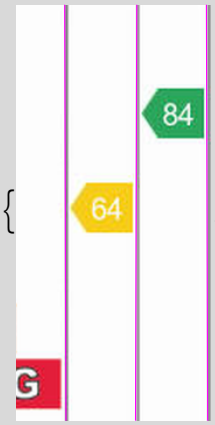
Out[89]=



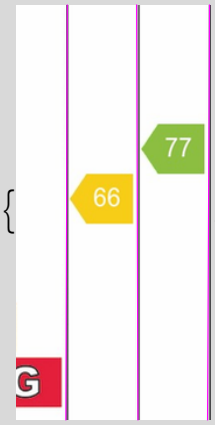
{{ , 5.45768 , 85.2375 , 165.146},



{ , 3.67876 , 84.9411 , 166.507 },



{ , 41.4479 , 103.523 , 163.472 },



{ , 43.6772 , 103.379 , 166.13 }}

cutting out rectangles based on x-coordinates of lines

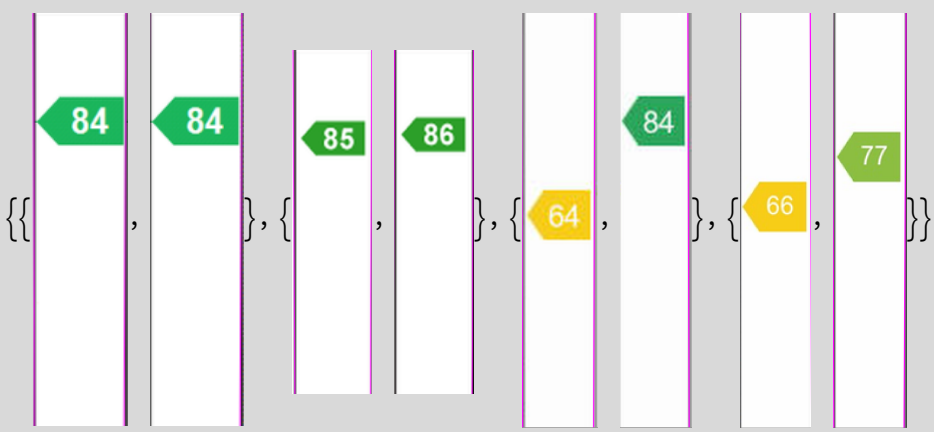
A check that the lines are equally spaced is also carried out.

Another check that the second line is near the edge should probably be carried out too

ImageDimensions could be hardcoded here.

```
fingersUniversal [{im_, x1_, x2_, x3_} /; (Abs[Abs[x1 - x2] - Abs[x2 - x3]] < 50)] := Module[
   {x, y, offset, fingerL, fingerR},
   {x, y} = ImageDimensions [im];
   offset = 0;
   fingerL = ImageTrim[im, {{x1 (x/x3), 1}, {x2 (x/x3), y}}];
   fingerR = ImageTrim[im, {{x2 (x/x3), 1}, {x3 (x/x3), y}}];
   {fingerL, fingerR}
  ];
```

Out[47]=



redoing output7 with the smaller more efficient images from output2

note that clusters 3 and 5 have rectangless 62 pixels wide, while 1 and 2 are 82 wide

In[91]:=

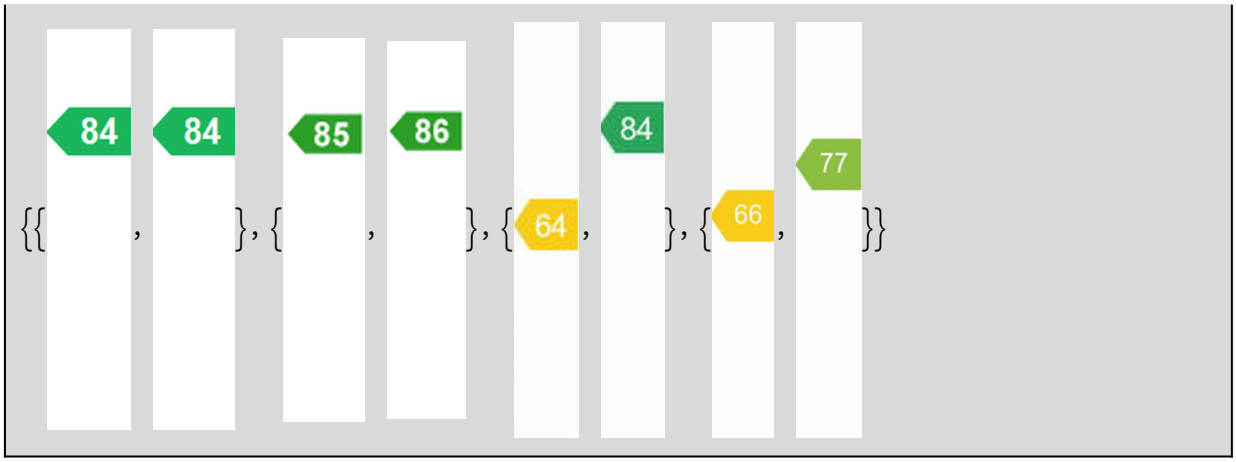
```
ImageDimensions /@ Flatten[output7 ]
```

Out[91]=

```
{{82, 357}, {81, 357}, {83, 357}, {82, 357}, {64, 357}, {62, 357}, {61, 357}, {64, 357}}
```

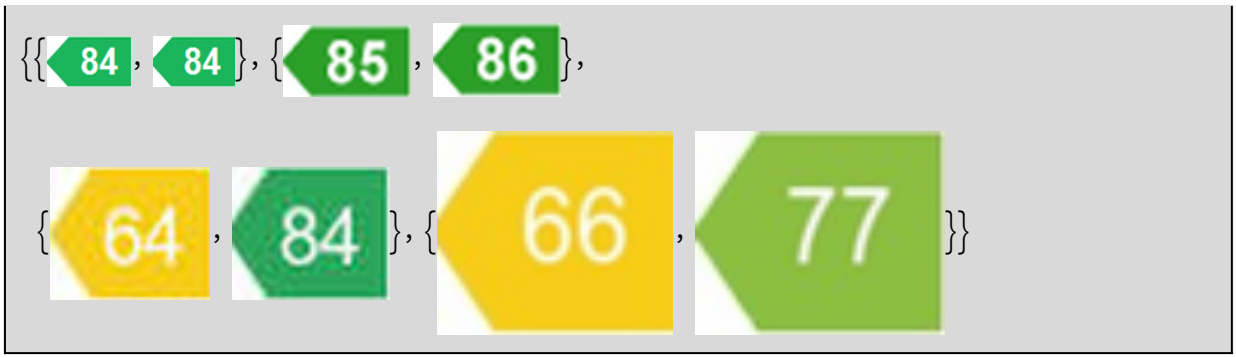
shaving 3 pixels off all sides

Out[105]=



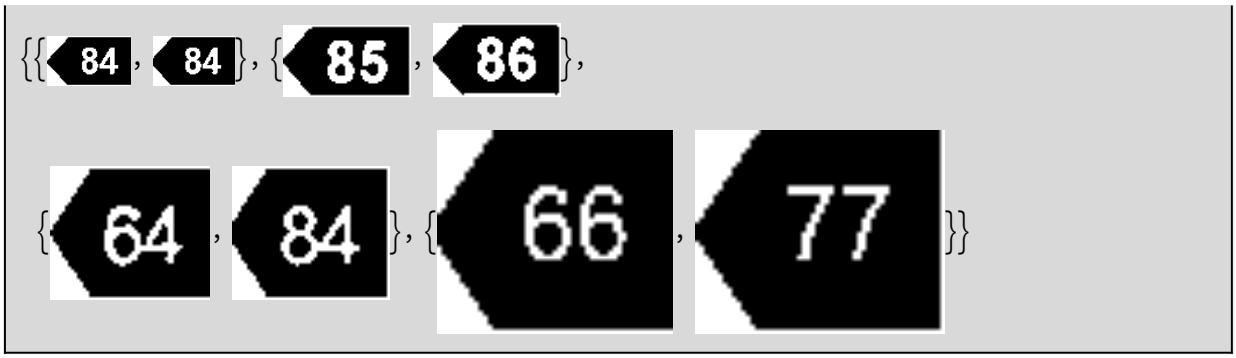
Cropping down to numbers

Out[119]=



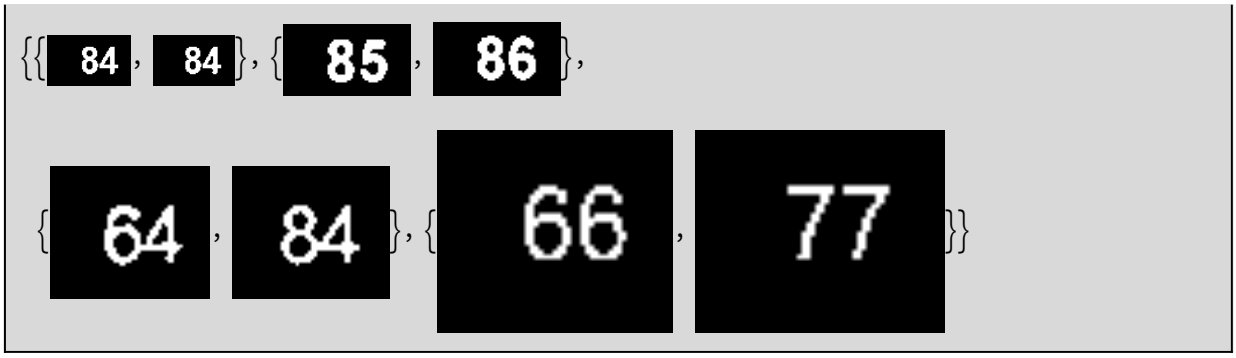
Binarizing

Out[121]=



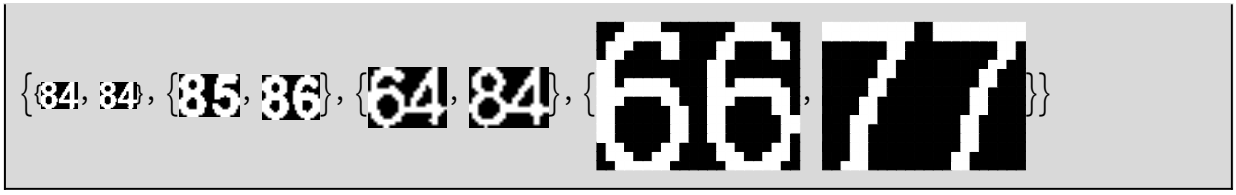
Deleting Border Components

Out[122]=



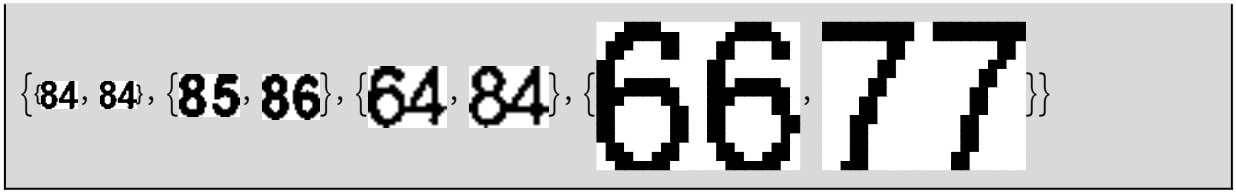
Autocropping

Out[124]=



Negative

Out[125]=



Adding a constant margin

Out[126]=

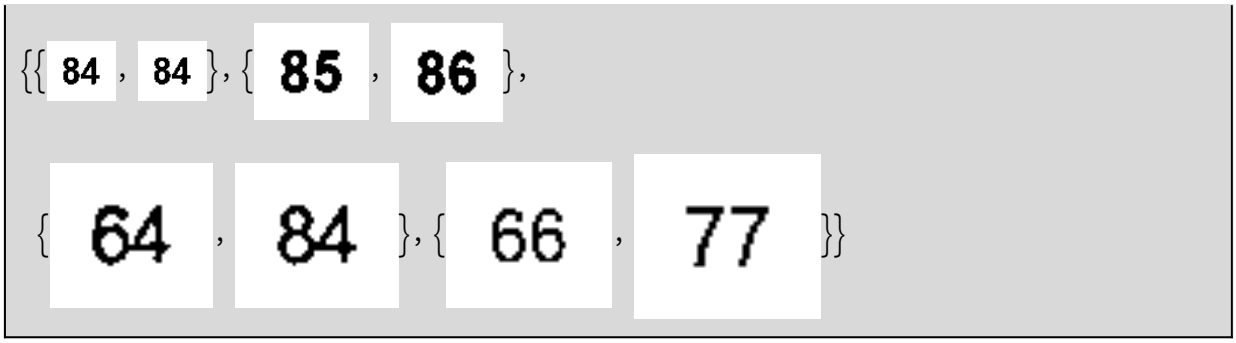


image size sanity check

In[127]:=
```
ImageDimensions /@ # & /@ {{84, 84}, {85, 86}, {64, 84}, {66, 77}}
```

Out[127]=
```
{{{59, 51}, {59, 51}}, {{59, 50}, {58, 51}}, {{54, 48}, {54, 48}}, {{50, 44}, {50, 44}}}
```

OCR output with confidence parameters

Out[129]=
```
{{{{8, 0.909306}, {4, 0.961772}}, {{8, 0.909306}, {4, 0.912496}}},
 {{{8, 0.878455}, {5, 0.898662}}, {{8, 0.891664}, {6, 0.8498}}},
 {{{6, 0.859835}, {4, 0.88489}}, {{8, 0.885921}, {4, 0.898897}}},
 {{{6, 0.908927}, {6, 0.886719}}, {{7, 0.924295}, {7, 0.878641}}}}
```

composing the characters into 2 digit numbers

In[130]:=
```
funky[lis_] := Module[{value, conf}, value = StringJoin[ToString /@ (First /@ lis)];
    conf = Mean[Last /@ lis];
    {value, conf}];
output16 = funky /@ # & /@ output15
```

Out[131]=
```
{{{84, 0.935539}, {84, 0.910901}}, {{85, 0.888558}, {86, 0.870732}},
 {{64, 0.872362}, {84, 0.892409}}, {{66, 0.897823}, {77, 0.901468}}}
```